

Bayesian Layers: A Module for Neural Network Uncertainty

Dustin Tran¹ Michael W. Dusenberry¹ Mark van der Wilk² Danijar Hafner¹

Abstract

We describe Bayesian Layers, a module designed for fast experimentation with neural network uncertainty. It extends neural network libraries with drop-in replacements for common layers. This enables composition via a unified abstraction over deterministic and stochastic functions and allows for scalability via the underlying system. These layers capture uncertainty over weights (Bayesian neural nets), pre-activation units (dropout), activations (“stochastic output layers”), or the function itself (Gaussian processes). They can also be reversible to propagate uncertainty from input to output. We include code examples for common architectures such as Bayesian LSTMs, deep GPs, and flow-based models. As demonstration, we fit a 5-billion parameter “Bayesian Transformer” on 512 TPUv2 cores for uncertainty in machine translation and a Bayesian dynamics model for model-based planning. Finally, we show how Bayesian Layers can be used within the Edward2 probabilistic programming language for probabilistic programs with stochastic processes.¹

1. Introduction

The rise of AI accelerators such as TPUs lets us utilize computation with 10^{16} FLOP/s and 4 TB of memory distributed across hundreds of processors (Jouppi et al., 2017). In principle, this lets us fit probabilistic models at many orders of magnitude larger than state of the art. We are particularly inspired by research on uncertainty-aware functions: priors and algorithms for Bayesian neural networks (e.g., Wen et al., 2018; Hafner et al., 2018b), scaling up Gaussian

¹Google Brain, Mountain View, California, USA ²Prowler.io, London, United Kingdom. *Work done as a Google AI Resident. Correspondence to: Dustin Tran <trandustin@google.com>.

¹All code is available at <https://github.com/tensorflow/tensor2tensor>. Dependency-wise, it extends Keras in TensorFlow (Chollet, 2016) and uses Edward2 (Tran et al., 2018) to operate with random variables. Namespaces: `import tensorflow as tf; ed=edward2`. Code snippets assume `tensorflow==1.12.0`.

```
batch_size = 256
features, labels = load_dataset(batch_size)
lstm = layers.VariationalLSTMCell(512)
output_layer = tf.keras.layers.Dense(labels.shape[-1])
state = lstm.get_initial_state(features)
nll = 0.
for t in range(features.shape[1]):
    net, state = lstm(features[:, t], state)
    logits = output_layer(net)
    nll += tf.losses.softmax_cross_entropy(
        onehot_labels=labels[:, t], logits=logits)
```

```
k1 = sum(lstm.losses) / batch_size
loss = nll + k1
optimizer = tf.train.AdamOptimizer()
train_op = optimizer.minimize(loss)
```

Figure 1: Bayesian RNN (Fortunato et al., 2017). Bayesian Layers integrate easily into existing workflows.

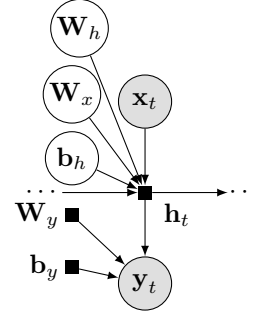


Figure 2: Graphical model depiction. Default arguments specify learnable distributions over the LSTM’s weights and biases; we apply a deterministic output layer.

processes (e.g., Salimbeni and Deisenroth, 2017; John and Hensman, 2018), and expressive distributions via invertible functions (e.g., Rezende and Mohamed, 2015).

Unfortunately, while research with uncertainty-aware functions are not limited by hardware, they are limited by software. Modern systems approach this by inventing a probabilistic programming language which encompasses all computable probability models as well as a universal inference engine (Goodman et al., 2012; Carpenter et al., 2016) or with composable inference (Tran et al., 2016; Bingham et al., 2018; Probtorch Developers, 2017). Alternatively, the software may use high-level abstractions in order to specify and fit specific model classes with a hand-derived algorithm (GPy, 2012; Vanhatalo et al., 2013; Matthews et al., 2017). These systems have all met success, but they tend

to be monolithic in design. This prevents research flexibility such as utilizing low-level communication primitives to truly scale up models to billions of parameters.

Most recently, Edward2 provides lower-level flexibility by enabling arbitrary numerical ops with random variables (Tran et al., 2018). However, it remains unclear how to leverage random variables for uncertainty-aware functions. For example, current practices with Bayesian neural networks require explicit network computation and variable management (Tran et al., 2016) or require indirection by intercepting weight instantiations of a deterministic layer (Bingham et al., 2018). Both designs are inflexible for many real-world uses in research. In practice, researchers often use the lower numerical level—without a unified design for uncertainty-aware functions as there are for deterministic neural networks and automatic differentiation. This forces researchers to reimplement even basic methods such as Bayes by Backprop (Blundell et al., 2015)—let alone build on and scale up more complex baselines.

This paper describes Bayesian Layers, an extension of neural network libraries which contributes one idea: instead of only deterministic functions as “layers”, enable distributions over functions. Bayesian Layers does not invent a new language. It inherits neural network semantics to specify uncertainty-aware models as a composition of layers. Each layer may capture uncertainty over weights (Bayesian neural nets), pre-activation units (dropout), activations (“stochastic output layers”), or the function itself (Gaussian processes). They can also be reversible layers that propagate uncertainty from input to output.

We include code examples for common architectures such as Bayesian LSTMs, deep GPs, and flow-based models. As demonstration, we fit a 5-billion parameter “Bayesian Transformer” on 512 TPUv2 cores for uncertainty in machine translation and a Bayesian dynamics model for model-based planning. Finally, we show how Bayesian Layers can be used as primitives in the Edward2 probabilistic programming language.

1.1. Related Work

There have been many software developments for distributions over functions. Our work takes classic inspiration from Radford Neal’s software in 1995 to enable flexible modeling with both Bayesian neural nets and GPs (Neal, 1995). Modern software typically focuses on only one of these directions. For Bayesian neural nets, researchers have commonly coupled variational sampling in neural net layers (e.g., code and algorithms from Gal and Ghahramani (2016); Louizos and Welling (2017)). For Gaussian processes, there have been significant developments in libraries (Rasmussen and Nickisch, 2010; GPy, 2012; Vanhatalo et al., 2013; Matthews et al., 2017; Al-Shedivat et al., 2017;

Gardner et al., 2018). Perhaps most similar to our work, Abolet (Abolet Developers, 2017) features variational BNNs and random feature approximations for GPs. Aside from API differences from all these works, our work tries to revive the spirit of enabling any function with uncertainty—whether that be, e.g., in the weights, activations, or the entire function—and to do so in a manner compatible with scalable deep learning ecosystems.

A related thread are probabilistic programming languages which build on the semantics of an existing functional programming language. Examples include HANSEI on OCaml, Church on Lisp, and Hakaru on Haskell (Kiselyov and Shan, 2009; Goodman et al., 2012; Narayanan et al., 2016). Neural network libraries can also be thought of as a (fairly simple) functional programming language, with limited higher-order logic and a type system of (finite lists of) n -dimensional arrays. Unlike the above probabilistic programming approaches, Bayesian Layers doesn’t introduce new primitives to the underlying language. As we describe next, it overloads the existing primitives with a method to handle randomness in any state in its execution.

2. Bayesian Layers

In neural network libraries, architectures decompose as a composition of “layer” objects as the core building block (Collobert et al., 2011; Al-Rfou et al., 2016; Jia et al., 2014; Chollet, 2016; Chen et al., 2015; Abadi et al., 2015; S. and N., 2016). These layers capture both the parameters and computation of a mathematical function into a programmable class.

In our work, we extend layers to capture “distributions over functions”, which we describe as a layer with uncertainty about some state in its computation—be it uncertainty in the weights, pre-activation units, activations, or the entire function. Each sample from the distribution instantiates a different function, e.g., a layer with a different weight configuration.

2.1. Bayesian Neural Network Layers

The Bayesian extension of any deterministic layer is to place a prior distribution over its weights and biases. These layers require several considerations. Figure 1 implements a Bayesian RNN.

Computing the integral We need to compute often-intractable integrals over weights and biases θ . Consider for example two cases, the variational objective for training and

```

class VariationalDense(tf.keras.layers.Dense):
    """Variational Bayesian dense layer."""
    def __init__(self,
                  units,
                  activation=None,
                  use_bias=True,
                  kernel_initializer='trainable_normal',
                  bias_initializer='zero',
                  kernel_regularizer='normal_kl_divergence',
                  bias_regularizer=None,
                  activity_regularizer=None,
                  **kwargs):
        super(VariationalDense, self).__init__(
            units=units,
            activation=activation,
            use_bias=use_bias,
            kernel_initializer=kernel_initializer,
            bias_initializer=bias_initializer,
            kernel_regularizer=kernel_regularizer,
            bias_regularizer=bias_regularizer,
            activity_regularizer=activity_regularizer,
            **kwargs)
    
```

Figure 3: Bayesian layers are modularized to fit existing neural net semantics of initializers, regularizers, and layers as they deem fit. Here, a Bayesian layer with reparameterization (Kingma and Welling, 2014; Blundell et al., 2015) is the same as its deterministic implementation. The only change is the default for `kernel_{initializer,regularizer}`; no additional methods are added.

the approximate predictive distribution for testing,

$$\begin{aligned}
 \text{ELBO}(\theta) &= \int q(\theta) \log p(\mathbf{y} \mid f_{\theta}(\mathbf{x})) d\theta \\
 &\quad - \text{KL}[q(\theta) \parallel p(\theta)], \\
 q(\mathbf{y} \mid \mathbf{x}) &= \int q(\theta) p(\mathbf{y} \mid f_{\theta}(\mathbf{x})) d\theta.
 \end{aligned}$$

Here, $\mathbf{x}$ may be a real-valued tensor as a batch of input features, $\mathbf{y}$ may be a vector-valued output for each feature set, and the function f encompasses the overall network as a composition of layers.

To enable different methods to estimate these integrals, we implement each estimator as its own Layer. The same Bayesian neural net can use entirely different computational graphs depending on the estimation (and therefore entirely different code). For example, sampling from $q(\theta)$ with reparameterization and running the deterministic layer computation is a generic way to evaluate layer-wise integrals (Kingma and Welling, 2014). Alternatively, given small weight dimensions, one could approximate each integral deterministically via quadrature. To enable modularity, the only restriction is that the overall integral estimation can be decomposed layer-wise; Section 4 describes such restrictions in more depth.

```

if FLAGS.be_bayesian:
    Conv2D = layers.VariationalConv2D
else:
    Conv2D = tf.keras.layers.Conv2D

model = tf.keras.Sequential([
    Conv2D(32, 5, 1, padding='SAME'),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Activation('relu'),
    Conv2D(32, kernel_size=5, strides=2, padding='SAME'),
    tf.keras.layers.BatchNormalization(),
    ...
])
    
```

Figure 4: Bayesian Layers are drop-in replacements for their deterministic counterparts.

Signature We’d like to have the Bayesian extension of a deterministic layer retain its mandatory constructor arguments as well as its call signature of tensor-dimensional inputs and tensor-dimensional outputs. This avoids cognitive overhead, letting one easily swap layers (Figure 4; Lauermann and Shridhar (2018)). For example, a dense (feed-forward) layer requires a `units` argument determining its output dimensionality; a convolutional layer also includes `kernel_size`. We maintain optional arguments (and add new ones) if they make sense.

Distributions over parameters To specify distributions, a natural idea is to overload the existing parameter initialization arguments in a Layer’s constructor; in Keras, it is `kernel_initializer` and `bias_initializer`. These arguments are extended to accept callables that take metadata such as input shape and return a distribution over the parameter. Distribution initializers may carry trainable parameters, each with their own initializers (pointwise or distributional). The default initializer represents a trainable approximate posterior in a variational inference scheme (Figure 3).

For the distribution abstraction, we use Edward RandomVariables (Tran et al., 2018). They are Tensors augmented with distribution methods such as `sample` and `log_prob`; by default, numerical ops operate on its sample Tensor. Layers perform forward passes using deterministic ops and the `RandomVariables`.

Distribution regularizers The variational training objective requires the evaluation of a KL term, which penalizes deviations of the learned $q(\theta)$ from the prior $p(\theta)$. Similar to distribution initializers, we overload the existing parameter regularization arguments in a layer’s constructor; in Keras, it is `kernel_regularizer` and `bias_regularizer` (Figure 3). These arguments are extended to accept callables that take in the kernel or bias `RandomVariables` and return a scalar Tensor. By default,

we use a KL divergence toward the standard normal distribution, which represents the penalty term common in variational Bayesian neural network training.

Explicitly returning regularizers in a Layer’s call ruins composability (see Signature above). Therefore Bayesian layers, like their deterministic counterparts, side-effect the computation: one queries an attribute to access any regularizers for, e.g., the loss function. [Figure 1](#) implements a Bayesian RNN; [Appendix A](#) implements a Bayesian CNN (ResNet-50).

2.2. Gaussian Process Layers

As opposed to representing distributions over functions through the weights, Gaussian processes represent distributions over functions by specifying the value of the function at different inputs. Recent advances have made Gaussian process inference computationally similar to Bayesian neural networks ([Hensman et al., 2013](#)). We only require a method to sample the function value at a new input, and evaluate KL regularizers. This allows GPs to be placed in the same framework as above.² [Figure 5](#) implements a deep GP.

The considerations are the same as above, and we make similar decisions:

Computing the integral We use a separate class for each estimator. This includes `GaussianProcess` for exact integration, which is only possible in limited situations; `SparseGaussianProcess` for inducing variable approximations; and `RandomFourierFeatures` for projection approximations.

Signature For the equivalent deterministic layer, maintain its mandatory arguments as well as tensor-dimensional inputs and outputs. For example, the number of units in a Gaussian process layer determine the GP’s output dimensionality, where `layers.GaussianProcess(32)` is the Bayesian nonparametric extension of `tf.keras.layers.Dense(32)`. Instead of an optional activation function argument, GP layers have optional mean and covariance function arguments which default to the zero function and squared exponential kernel respectively. We also include an optional argument for what set of inputs and outputs to condition on: this allows the GP layer to perform both prior and posterior predictive computations. Any state in the layer’s computational graph may be trainable—whether they be kernel hyperparameters or the inputs and outputs that function conditions on.

²More broadly, these ideas extend to stochastic processes. For example, we plan to implement a Poisson process layer for scalable point process modeling.

```
batch_size = 256
features, labels = load_spatial_data(batch_size)

model = tf.keras.Sequential([
    tf.keras.layers.Flatten(), # no spatial knowledge
    layers.SparseGaussianProcess(units=256,
                                num_inducing=512),
    layers.SparseGaussianProcess(units=256,
                                num_inducing=512),
    layers.SparseGaussianProcess(units=10,
                                num_inducing=512),
])
predictions = model(features)
neg_log_likelihood = tf.losses.mean_squared_error(
    labels=labels, predictions=predictions)
kl = sum(model.losses) / batch_size
loss = neg_log_likelihood + kl
train_op = tf.train.AdamOptimizer().minimize(loss)
```

Figure 5: Three-layer deep GP with variational inference ([Salimbeni and Deisenroth, 2017](#); [Damianou and Lawrence, 2013](#)). We apply it for regression given batches of spatial inputs and vector-valued outputs. We flatten inputs to use the default squared exponential kernel; this naturally extends to pass in a more sophisticated kernel function.

Distribution regularizers By default, we include no regularizer for exact GPs, a KL divergence regularizer on the inducing output distribution for sparse GPs, and a KL divergence regularizer on weights for random projection approximations. These defaults reflect each inference method’s standard for training, where the KL regularizers use the same implementation as the Bayesian neural nets’.

2.3. Stochastic Output Layers

In addition to uncertainty over the *mapping* defined by a layer, we may want to simply add stochasticity to the output. These outputs have a tractable distribution, and we often would like to access its properties: for example, maximum likelihood with an autoregressive network whose output is a discretized logistic mixture ([Salimans et al., 2017](#)) ([Figure 6](#)); or an auto-encoder with stochastic encoders and decoders ([Figure 7](#)).³

Signature To implement stochastic output layers, we perform deterministic computations given a tensor-dimensional input and return a `RandomVariable`. Because `RandomVariables` are Tensor-like objects, one can operate on them as if they were Tensors: composing stochastic output layers is valid. In addition, using such a layer as the last one in a network allows one to compute properties such as a network’s entropy or likelihood given

³ In previous figures, we used loss functions such as `mean_squared_error`. With stochastic output layers, we can replace them with a layer returning the likelihood and calling `log_prob`.


```

def build_image_transformer(hparams):
    x = tf.keras.layers.Input(shape=input_shape)
    x = ChannelEmbedding(hparams.hidden_size)(x)
    x = tf.pad(x, [[0, 0], [1, 0], [0, 0]][: , :-1, :])
    x = tf.keras.layers.Conv2DTranspose, activation=tf.nn.relu)
    x = PositionalEmbedding(max_length=128*128*3)(x)
    x = tf.keras.layers.Dropout(0.3)(x)
    for _ in range(hparams.num_layers):
        y = MaskedLocalAttention1D(hparams)(x)
        x = LayerNormalization()(
            tf.keras.layers.Dropout(0.3)(y) + x)
        y = tf.keras.layers.Dense(
            x, hparams.filter_size, activation=tf.nn.relu)
        y = tf.keras.layers.Dense(
            hparams.hidden_size, activation=None)(y)
        x = LayerNormalization()(
            tf.keras.layers.Dropout(0.3)(y) + x)
    x = layers.MixtureofLogistic(
        3, num_components=5)(x)
    x = layers.Discretize(x)
    model = tf.keras.Model(inputs=inputs,
                           outputs=x,
                           name='ImageTransformer')

    return model

transformer = build_image_transformer(hparams)
loss = -tf.reduce_sum(
    transformer(features).distribution.log_prob(
        features))
train_op = tf.train.AdamOptimizer().minimize(loss)

Conv2D = functools.partial(
    tf.keras.layers.Conv2D, activation=tf.nn.relu)
Conv2DTranspose = functools.partial(
    tf.keras.layers.Conv2DTranspose, activation=tf.nn.relu)

encoder = tf.keras.Sequential([
    Conv2D(128, 5, 1, padding='SAME'),
    Conv2D(128, 5, 2, padding='SAME'),
    Conv2D(256, 5, 2, padding='SAME'),
    Conv2D(256, 5, 2, padding='SAME'),
    Conv2D(512, 7, 1, padding='VALID'),
    layers.Normal(name='latent_code'),
])
decoder = tf.keras.Sequential([
    Conv2DTranspose(256, 7, 1, padding='VALID'),
    Conv2DTranspose(256, 5, 2, padding='SAME'),
    Conv2DTranspose(128, 5, 2, padding='SAME'),
    Conv2DTranspose(128, 5, 2, padding='SAME'),
    Conv2DTranspose(128, 5, 1, padding='SAME'),
    Conv2D(3*256, 5, 1, padding='SAME', activation=None),
    tf.keras.layers.Reshape([256, 256, 3, 256]),
    layers.Categorical(name='image'),
])
encoding = encoder(features)
nll = decoder(encoding).distribution.log_prob(features)
kl = encoding.distribution.kl_divergence(ed.Normal(0., 1.))
loss = tf.reduce_mean(nll + kl)
train_op = tf.train.AdamOptimizer().minimize(loss)
    
```

Figure 6: Image Transformer with discretized logistic mixture (Parmar et al., 2018) over 128x128x3 features. Stochastic output layers let one easily experiment with the likelihood. We assume layers which don’t exist in Keras; functional versions are available in Tensor2Tensor (Vaswani et al., 2018).

data.

Stochastic output layers typically don’t have mandatory constructor arguments. An optional `units` argument determines its output dimensionality (operated on via a trainable linear projection); the default maintains the input shape and has no such projection.

2.4. Reversible Layers

With random variables in layers, one can naturally capture invertible neural networks which propagate uncertainty from input to output. This allows one to perform transformations of random variables, ranging from simple transformations such as for a log-normal distribution or high-dimensional transformations for flow-based models.

We make two considerations to design reversible layers:

Inversion Invertible neural networks are not possible with current libraries. A natural idea is to design a new abstraction for invertible functions (Dillon et al., 2017). Un-

Figure 7: A variational auto-encoder for compressing 256x256x3 ImageNet into a 32x32x3 latent code. Stochastic output layers are a natural approach for specifying stochastic encoders and decoders, and utilizing their log-probability or KL divergence.

fortunately, this prevents interoperability with existing layer and model abstractions. Instead, we simply overload the notion of a “layer” by adding an additional method `reverse` which performs the inverse computation of its call and optionally `log_det_jacobian`. A higher-order layer called `layers.Reverse` takes a layer as input and returns another layer swapping the forward and reverse computation; by ducktyping, the reverse layer raises an error only during its call if `reverse` is not implemented. Avoiding a new abstraction both simplifies usage and also makes reversible layers compatible with other higher-order layers such as `tf.keras.Sequential`, which returns a composition of a sequence of layers.

Propagating Uncertainty As with other deterministic layers, reversible layers take a tensor-dimensional input and return a tensor-dimensional output where the output dimensionality is determined by its arguments. In order to propagate uncertainty from input to output, reversible layers may also take a `RandomVariable` as input and return a transformed `RandomVariable` determined by its call, re-

```

batch_size = 256
features = load_cifar10(batch_size)

model = tf.keras.Sequential([
    layers.RealNVP(MADE(hidden_dims=[512, 512])),
    layers.RealNVP(MADE(hidden_dims=[512, 512],
                          order='right-to-left')),
    layers.RealNVP(MADE(hidden_dims=[512, 512])),
])
base = ed.Normal(tf.zeros([batch_size, 32*32*3]),
                1.)
outputs = model(base)
loss = -tf.reduce_sum(outputs.distribution.log_prob(
    features))
train_op = tf.train.AdamOptimizer().minimize(loss)
    
```

Figure 8: A flow-based model for image generation (Dinh et al., 2017).

verse, and $\log_{\det} \text{jacobian}$.⁴ Figure 8 implements RealNVP (Dinh et al., 2017), which is a reversible layer parameterized by another network (here, MADE (Germain et al., 2015)). These ideas also extend to reversible networks that enable backpropagation without storing intermediate activations in memory during the forward pass (Gomez et al., 2017).

2.5. Layers for Probabilistic Programming

While the framework we laid out so far tightly integrates deep Bayesian modelling into existing ecosystems, we have deliberately limited our scope. In particular, our layers tie the model specification to the inference algorithm (typically, variational inference). Bayesian Layers’ core assumption is the modularization of inference per layer. This makes inference procedures which depend on the full parameter space, such as Markov chain Monte Carlo, difficult to fit within the framework.

Figure 9 shows how one can utilize Bayesian Layers in the Edward2 probabilistic programming language for more flexible modeling and inference. We could use, e.g., expectation propagation (Bui et al., 2016; Hernández-Lobato and Adams, 2015). See Tran et al. (2018) for details on how to use Edward2’s tracing mechanisms for arbitrary training.

3. Experiments

We described a design for uncertainty-aware models built on top of neural network libraries. In experiments, we illustrate two points: 1. Bayesian Layers is efficient and makes possible new model classes that haven’t been tried before (in either scale or flexibility); and 2. utilizing such Bayesian

⁴We implement layers.Discretize this way in Figure 6. It takes a continuous RandomVariable as input and returns a transformed variable with probabilities integrated over bins.

```

def model(input_shape):
    """Spatial point process with latent intensity."""
    rate = tf.keras.Sequential([
        layers.GaussianProcess(64),
        layers.GaussianProcess(input_shape),
        tf.keras.layers.Activation('softplus'),
    ])
    return layers.PoissonProcess(rate=rate)

def posterior():
    """Posterior approximation of intensity."""
    rate = tf.keras.Sequential([
        layers.SparseGaussianProcess(units=64,
                                      num_inducing=512),
        layers.SparseGaussianProcess(units=1,
                                      num_inducing=512),
        tf.keras.layers.Activation('softplus'),
    ])
    return rate
    
```

Figure 9: Cox process with a deep GP prior and a sparse GP posterior approximation. Using Bayesian Layers in a probabilistic programming language allows for a clean distinction between modeling and inference, as well as more flexible inference algorithms.

models provides benefits in applications including model-based planning.

3.1. Model-Parallel Bayesian Transformer for Machine Translation

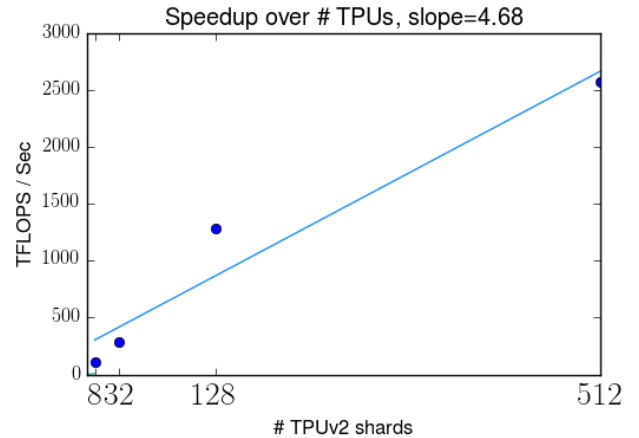


Figure 10: Bayesian Transformer implemented with model parallelism ranging from 8 TPUv2 shards (core) to 512. As desired, the model’s training performance scales linearly as the number of cores increases.

We implemented a “Bayesian Transformer” for the WMT14 EN-FR translation task. Using Mesh TensorFlow (Shazeer et al., 2018), we took a 2.8 billion parameter Transformer which reports a state-of-the-art BLEU score of 43.9. We

then augmented the model with priors over the projection matrices by replacing calls to a multihead-attention layer with its Bayesian counterpart (using the Flipout estimator); we also made the pointwise feedforward layers Bayesian. Figure 10 shows that we can fit models with over 5-billion parameters (roughly twice as many due to a mean and standard deviation parameter), utilizing up to 2500 TFLOPs on 512 TPUv2 cores.

In attempting these scales, we were able to reach state-of-the-art perplexities while achieving a higher predictive variance. This may suggest the Bayesian Transformer more correctly accounts for uncertainty given that the dataset is actually fairly small given the size of the model. We also implemented a “Bayesian Transformer” for the One-Billion-Word Language Modeling Benchmark (Chelba et al., 2013), maintaining the same state-of-the-art perplexity of 23.1. We identified a number of challenges in both scaling up Bayesian neural nets and understanding their text applications; we leave this for future work separate from this systems paper.

3.2. Bayesian Dynamics Model for Model-Based Reinforcement Learning

In reinforcement learning, uncertainty estimates can allow for directed exploration, safe exploration, and robust control. Still relatively few works leverage deep Bayesian models for control (Gal et al., 2016; Azizzadenesheli et al., 2018). We argue that this might be because implementing and training these models can be difficult and time consuming. To demonstrate our module, we implement Bayesian PlaNet, based on the work of Hafner et al. (2018a). The original PlaNet agent learns a latent dynamics model as a sequential VAE on image observations. A sample-based planner then searches for the most promising action sequence in the latent space of the model.

We extend this agent by changing the fully connected layers of the transition function to their Bayesian counterparts, VariationalDense. Swapping the layers and adding the KL term to the loss, we reach a score of 614 on the cheetah task, matching the performance of the original agent. We monitor the KL divergence of the weight posterior to verify that the model indeed learns a non-trivial belief. This result demonstrates that incorporating model estimates into reinforcement learning agents can be straightforward given the right software abstractions. The fact that the same performance is achieved opens up many possible approaches for exploration and robust control; see Appendix B.

4. Discussion

We described Bayesian Layers, a module designed for fast experimentation with neural network uncertainty. By cap-

turing uncertainty-aware functions, Bayesian Layers lets one naturally experiment with and scale up Bayesian neural networks, GPs, and flow-based models.

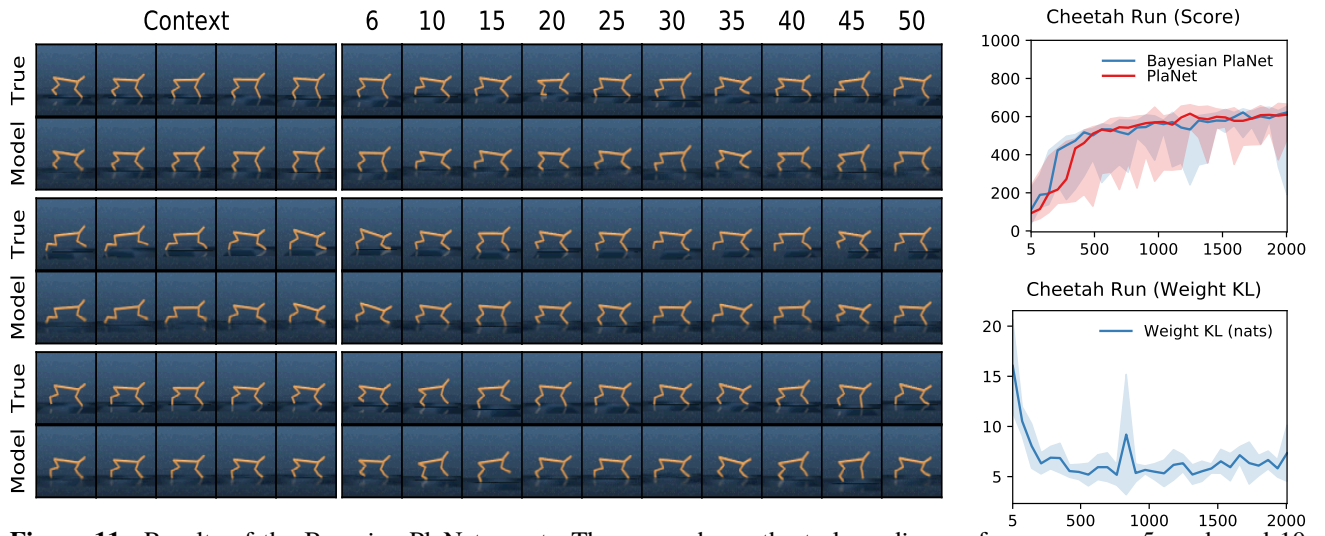


Figure 11: Results of the Bayesian PlaNet agent. The score shows the task median performance over 5 seeds and 10 episodes each, with percentiles 5 to 95 shaded. Our Bayesian version of the method reaches the same task performance. The graph of the weight KL shows that the weight posterior learns a non-trivial function. The open-loop video predictions show that the agent can accurately make predictions into the future for 50 time steps.

References

- Abadi, M., Agarwal, A., Barham, P., Brevdo, E., Chen, Z., Citro, C., Corrado, G. S., Davis, A., Dean, J., Devin, M., Ghemawat, S., Goodfellow, I., Harp, A., Irving, G., Isard, M., Jia, Y., Jozefowicz, R., Kaiser, L., Kudlur, M., Levenberg, J., Mané, D., Monga, R., Moore, S., Murray, D., Olah, C., Schuster, M., Shlens, J., Steiner, B., Sutskever, I., Talwar, K., Tucker, P., Vanhoucke, V., Vasudevan, V., Viégas, F., Vinyals, O., Warden, P., Wattemberg, M., Wicke, M., Yu, Y., and Zheng, X. (2015). TensorFlow: Large-scale machine learning on heterogeneous systems. Software available from tensorflow.org.
- Aboleth Developers (2017). Aboleth. <https://github.com/data61/aboleth>.
- Al-Rfou, R., Alain, G., Almahairi, A., Angermueller, C., Bahdanau, D., Ballas, N., Bastien, F., Bayer, J., Belikov, A., Belopolsky, A., Bengio, Y., Bergeron, A., Bergstra, J., Bisson, V., Blecher Snyder, J., Bouchard, N., Boulanger-Lewandowski, N., Bouthillier, X., de Brébisson, A., Breuleux, O., Carrier, P.-L., Cho, K., Chorowski, J., Christiano, P., Cooijmans, T., Côté, M.-A., Côté, M., Courville, A., Dauphin, Y. N., Delalleau, O., Demouth, J., Desjardins, G., Dieleman, S., Dinh, L., Ducoffe, M., Dumoulin, V., Ebrahimi Kahou, S., Erhan, D., Fan, Z., Firat, O., Germain, M., Glorot, X., Goodfellow, I., Graham, M., Gulcehre, C., Hamel, P., Harlouchet, I., Heng, J.-P., Hidasi, B., Honari, S., Jain, A., Jean, S., Jia, K., Korobov, M., Kulkarni, V., Lamb, A., Lamblin, P., Larsen, E., Laurent, C., Lee, S., Lefrancois, S., Lemieux, S., Léonard, N., Lin, Z., Livezey, J. A., Lorenz, C., Lowin, J., Ma, Q., Manzagol, P.-A., Mastropietro, O., McGibbon, R. T., Memisevic, R., van Merriënboer, B., Michalski, V., Mirza, M., Orlandi, A., Pal, C., Pascanu, R., Pezeshki, M., Raffel, C., Renshaw, D., Rocklin, M., Romero, A., Roth, M., Sadowski, P., Salvatier, J., Savard, F., Schlüter, J., Schulman, J., Schwartz, G., Serban, I. V., Serdyuk, D., Shabanian, S., Simon, E., Spieckermann, S., Subramanyam, S. R., Sygnowski, J., Tanguay, J., van Tulder, G., Turian, J., Urban, S., Vincent, P., Visin, F., de Vries, H., Warde-Farley, D., Webb, D. J., Willson, M., Xu, K., Xue, L., Yao, L., Zhang, S., and Zhang, Y. (2016). Theano: A Python framework for fast computation of mathematical expressions. *arXiv preprint arXiv:1605.02688*.
- Al-Shedivat, M., Wilson, A. G., Saatchi, Y., Hu, Z., and Xing, E. P. (2017). Learning scalable deep kernels with recurrent structure. *Journal of Machine Learning Research*, 18(1).
- Azizzadenesheli, K., Brunskill, E., and Anandkumar, A. (2018). Efficient exploration through bayesian deep q-networks. *arXiv preprint arXiv:1802.04412*.
- Bingham, E., Chen, J. P., Jankowiak, M., Obermeyer, F., Pradhan, N., Karaletsos, T., Singh, R., Szerlip, P., Horsfall, P., and Goodman, N. D. (2018). Pyro: Deep Universal Probabilistic Programming. *arXiv preprint arXiv:1810.09538*.
- Blundell, C., Cornebise, J., Kavukcuoglu, K., and Wierstra, D. (2015). Weight uncertainty in neural networks. In *International Conference on Machine Learning*.
- Bui, T., Hernandez-Lobato, D., Hernandez-Lobato, J., Li, Y., and Turner, R. (2016). Deep gaussian processes for regression using approximate expectation propagation. In Balcan, M. F. and Weinberger, K. Q., editors, *Proceedings of The 33rd International Conference on Machine Learning*, volume 48 of *Proceedings of Machine Learning Research*, pages 1472–1481, New York, New York, USA. PMLR.
- Carpenter, B., Gelman, A., Hoffman, M. D., Lee, D., Goodrich, B., Betancourt, M., Brubaker, M., Guo, J., Li, P., and Riddell, A. (2016). Stan: A probabilistic programming language. *Journal of Statistical Software*.
- Chelba, C., Mikolov, T., Schuster, M., Ge, Q., Brants, T., and Koehn, P. (2013). One billion word benchmark for measuring progress in statistical language modeling. *CoRR*, abs/1312.3005.
- Chen, T., Li, M., Li, Y., Lin, M., Wang, N., Wang, M., Xiao, T., Xu, B., Zhang, C., and Zhang, Z. (2015). MXNet: A flexible and efficient machine learning library for heterogeneous distributed systems. *arXiv preprint arXiv:1512.01274*.

- Chollet, F. (2016). Keras. <https://github.com/fchollet/keras>.
- Collobert, R., Kavukcuoglu, K., and Farabet, C. (2011). Torch7: A matlab-like environment for machine learning. In *BigLearn, NIPS Workshop*.
- Damianou, A. and Lawrence, N. (2013). Deep gaussian processes. In *Artificial Intelligence and Statistics*, pages 207–215.
- Dillon, J. V., Langmore, I., Tran, D., Brevdo, E., Vasudevan, S., Moore, D., Patton, B., Alemi, A., Hoffman, M., and Saurous, R. A. (2017). TensorFlow Distributions. *arXiv preprint arXiv:1711.10604*.
- Dinh, L., Sohl-Dickstein, J., and Bengio, S. (2017). Density estimation using real nvp. In *International Conference on Learning Representations*.
- Fortunato, M., Blundell, C., and Vinyals, O. (2017). Bayesian recurrent neural networks. *arXiv preprint arXiv:1704.02798*.
- Gal, Y. and Ghahramani, Z. (2016). Dropout as a bayesian approximation: Representing model uncertainty in deep learning. In *international conference on machine learning*, pages 1050–1059.
- Gal, Y., McAllister, R., and Rasmussen, C. E. (2016). Improving pilco with bayesian neural network dynamics models. In *Data-Efficient Machine Learning workshop, ICML*, volume 4.
- Gardner, J. R., Pleiss, G., Bindel, D., Weinberger, K. Q., and Wilson, A. G. (2018). Gpytorch: Blackbox matrix-matrix gaussian process inference with gpu acceleration. In *NeurIPS*.
- Germain, M., Gregor, K., Murray, I., and Larochelle, H. (2015). Made: Masked autoencoder for distribution estimation. In *International Conference on Machine Learning*, pages 881–889.
- Gomez, A. N., Ren, M., Urtasun, R., and Grosse, R. B. (2017). The reversible residual network: Backpropagation without storing activations. In *Neural Information Processing Systems*.
- Goodman, N., Mansinghka, V., Roy, D. M., Bonawitz, K., and Tenenbaum, J. B. (2012). Church: a language for generative models. *arXiv preprint arXiv:1206.3255*.
- GPY (since 2012). GPY: A gaussian process framework in python. <http://github.com/SheffieldML/GPY>.
- Hafner, D., Lillicrap, T., Fischer, I., Villegas, R., Ha, D., Lee, H., and Davidson, J. (2018a). Learning latent dynamics for planning from pixels. *arXiv preprint arXiv:1811.04551*.
- Hafner, D., Tran, D., Irpan, A., Lillicrap, T., and Davidson, J. (2018b). Reliable uncertainty estimates in deep neural networks using noise contrastive priors. *arXiv preprint*.
- Hensman, J., Fusi, N., and Lawrence, N. D. (2013). Gaussian processes for big data. In *Conference on Uncertainty in Artificial Intelligence*.
- Hernández-Lobato, J. M. and Adams, R. P. (2015). Probabilistic backpropagation for scalable learning of bayesian neural networks. In *Proceedings of the 32Nd International Conference on International Conference on Machine Learning - Volume 37, ICML’15*, pages 1861–1869. JMLR.org.
- Jia, Y., Shelhamer, E., Donahue, J., Karayev, S., Long, J., Girshick, R., Guadarrama, S., and Darrell, T. (2014). Caffe: Convolutional architecture for fast feature embedding. In *Proceedings of the 22nd ACM international conference on Multimedia*, pages 675–678. ACM.
- John, S. T. and Hensman, J. (2018). Large-scale cox process inference using variational fourier features. *arXiv preprint arXiv:1804.01016*.
- Jouppi, N. P., Young, C., Patil, N., Patterson, D., Agrawal, G., Bajwa, R., Bates, S., Bhatia, S., Boden, N., Borchers, A., et al. (2017). In-datacenter performance analysis of a tensor processing unit. In *Proceedings of the 44th Annual International Symposium on Computer Architecture*.
- Kingma, D. P. and Welling, M. (2014). Auto-encoding variational Bayes. In *International Conference on Learning Representations*.
- Kiselyov, O. and Shan, C.-C. (2009). Embedded probabilistic programming. In *DSL*, volume 5658, pages 360–384. Springer.
- Laumann, F. and Shridhar, K. (2018). Bayesian convolutional neural networks. *arXiv preprint arXiv:1806.05978*.
- Louizos, C. and Welling, M. (2017). Multiplicative normalizing flows for variational bayesian neural networks. *arXiv preprint arXiv:1703.01961*.
- Matthews, A. G. d. G., van der Wilk, M., Nickson, T., Fujii, K., Boukouvalas, A., León-Villagrà, P., Ghahramani, Z., and Hensman, J. (2017). GPflow: A Gaussian process library using TensorFlow. *Journal of Machine Learning Research*, 18(40):1–6.
- Narayanan, P., Carette, J., Romano, W., Shan, C.-c., and Zinkov, R. (2016). Probabilistic Inference by Program Transformation in Hakaru (System Description). In *International Symposium on Functional and Logic Programming*, pages 62–79, Cham. Springer, Cham.

- Neal, R. (1995). Software for flexible bayesian modeling and markov chain sampling. <https://www.cs.toronto.edu/~radford/fbm.software.html>.
- Parmar, N., Vaswani, A., Uszkoreit, J., Kaiser, Ł., Shazeer, N., Ku, A., and Tran, D. (2018). Image transformer. In *International Conference on Machine Learning*.
- Probtorch Developers (2017). Probtorch. <https://github.com/probtorch/probtorch>.
- Rasmussen, C. E. and Nickisch, H. (2010). Gaussian processes for machine learning (gpml) toolbox. *Journal of machine learning research*, 11(Nov):3011–3015.
- Rezende, D. J. and Mohamed, S. (2015). Variational inference with normalizing flows. In *International Conference on Machine Learning*.
- S., G. and N., S. (2016). TensorFlow-Slim: A lightweight library for defining, training and evaluating complex models in TensorFlow.
- Salimans, T., Karpathy, A., Chen, X., and Kingma, D. P. (2017). PixelCNN++: Improving the pixelcnn with discretized logistic mixture likelihood and other modifications. *arXiv preprint arXiv:1701.05517*.
- Salimbeni, H. and Deisenroth, M. (2017). Doubly stochastic variational inference for deep gaussian processes. In *Advances in Neural Information Processing Systems*, pages 4588–4599.
- Shazeer, N., Cheng, Y., Parmar, N., Tran, D., Vaswani, A., Koanantakool, P., Hawkins, P., Lee, H., Hong, M., Young, C., Sepassi, R., and Hechtman, B. (2018). MeshTensorFlow: Deep learning for supercomputers. In *Neural Information Processing Systems*.
- Tran, D., Hoffman, M. D., Moore, D., Suter, C., Vasudevan, S., Radul, A., Johnson, M., and Saurous, R. A. (2018). Simple, distributed, and accelerated probabilistic programming. In *Neural Information Processing Systems*.
- Tran, D., Kucukelbir, A., Dieng, A. B., Rudolph, M., Liang, D., and Blei, D. M. (2016). Edward: A library for probabilistic modeling, inference, and criticism. *arXiv preprint arXiv:1610.09787*.
- Vanhatalo, J., Riihimäki, J., Hartikainen, J., Jylänki, P., Tolvanen, V., and Vehtari, A. (2013). Gpstuff: Bayesian modeling with gaussian processes. *Journal of Machine Learning Research*, 14(Apr):1175–1179.
- Vaswani, A., Bengio, S., Brevdo, E., Chollet, F., Gomez, A. N., Gouws, S., Jones, L., Kaiser, L., Kalchbrenner, N., Parmar, N., Sepassi, R., Shazeer, N., and Uszkoreit, J. (2018). Tensor2tensor for neural machine translation. *CoRR*, abs/1803.07416.
- Wen, Y., Vicol, P., Ba, J., Tran, D., and Grosse, R. (2018). Flipout: Efficient pseudo-independent weight perturbations on mini-batches. In *International Conference on Learning Representations*.

A. Bayesian ResNet-50

See Figure 12.

```
def conv_block(inputs, kernel_size, filters, strides=(2, 2)):
    filters1, filters2, filters3 = filters
    x = layers.VariationalConv2D(filters1, (1, 1),
                                strides=strides)(inputs)
    x = tf.keras.layers.BatchNormalization()(x)
    x = tf.keras.layers.Activation('relu')(x)
    x = layers.VariationalConv2D(filters2, kernel_size,
                                padding='SAME')(x)
    x = tf.keras.layers.BatchNormalization()(x)
    x = tf.keras.layers.Activation('relu')(x)
    x = layers.VariationalConv2D(filters3, (1, 1))(x)
    x = tf.keras.layers.BatchNormalization()(x)
    shortcut = layers.VariationalConv2D(filters3, (1, 1),
                                        strides=strides)(inputs)
    shortcut = tf.keras.layers.BatchNormalization()(shortcut)
    x = tf.keras.layers.add([x, shortcut])
    x = tf.keras.layers.Activation('relu')(x)
    return x

def identity_block(inputs, kernel_size, filters):
    filters1, filters2, filters3 = filters
    x = layers.VariationalConv2D(filters1, (1, 1))(inputs)
    x = tf.keras.layers.BatchNormalization()(x)
    x = tf.keras.layers.Activation('relu')(x)
    x = layers.VariationalConv2D(filters2, kernel_size,
                                padding='SAME')(x)
    x = tf.keras.layers.BatchNormalization()(x)
    x = tf.keras.layers.Activation('relu')(x)
    x = layers.VariationalConv2D(filters3, (1, 1))(x)
    x = tf.keras.layers.BatchNormalization()(x)
    x = tf.keras.layers.add([x, inputs])
    x = tf.keras.layers.Activation('relu')(x)
    return x

def build_bayesian_resnet50(input_shape=None,
                             num_classes=1000):
    inputs = tf.keras.layers.Input(shape=input_shape,
                                    dtype='float32')
    x = tf.keras.layers.ZeroPadding2D((3, 3))(inputs)
    x = layers.VariationalConv2D(64, (7, 7),
                                strides=(2, 2), padding='VALID')(x)
    x = tf.keras.layers.BatchNormalization()(x)
    x = tf.keras.layers.Activation('relu')(x)
    x = tf.keras.layers.ZeroPadding2D((1, 1))(x)
    x = tf.keras.layers.MaxPooling2D((3, 3), strides=(2, 2))(x)
    x = conv_block(x, 3, [64, 64, 256], strides=(1, 1))
    x = identity_block(x, 3, [64, 64, 256])
    x = conv_block(x, 3, [128, 128, 512])
    x = identity_block(x, 3, [128, 128, 512])
    x = identity_block(x, 3, [128, 128, 512])
    x = identity_block(x, 3, [128, 128, 512])
    x = conv_block(x, 3, [256, 256, 1024])
    x = identity_block(x, 3, [256, 256, 1024])
    x = identity_block(x, 3, [256, 256, 1024])
    x = identity_block(x, 3, [256, 256, 1024])
    x = identity_block(x, 3, [256, 256, 1024])
    x = conv_block(x, 3, [512, 512, 2048])
    x = identity_block(x, 3, [512, 512, 2048])
    x = identity_block(x, 3, [512, 512, 2048])
    x = tf.keras.layers.GlobalAveragePooling2D()(x)
    x = layers.VariationalDense(num_classes)(x)
    model = models.Model(inputs, x, name='resnet50')
    return model

bayesian_resnet50 = build_bayesian_resnet50()
logits = bayesian_resnet50(features)
neg_log_likelihood = tf.losses.sparse_softmax_cross_entropy(
    labels=labels, logits=logits, reduction=tf.losses.reduction.MEAN)
kl = sum(bayesian_resnet50.losses) / batch_size # KL are Layer side-effects
loss = neg_log_likelihood + kl
train_op = tf.train.AdamOptimizer().minimize(loss)

# Alternatively, run the following instead of a manual train_op.
model.compile(optimizer=tf.train.AdamOptimizer(),
              loss='categorical_crossentropy',
              metrics=['accuracy'])
model.fit(features, labels, batch_size=32, epochs=5)
```

Figure 12: Bayesian ResNet-50.

B. Bayesian PlaNet

See Figure 13.

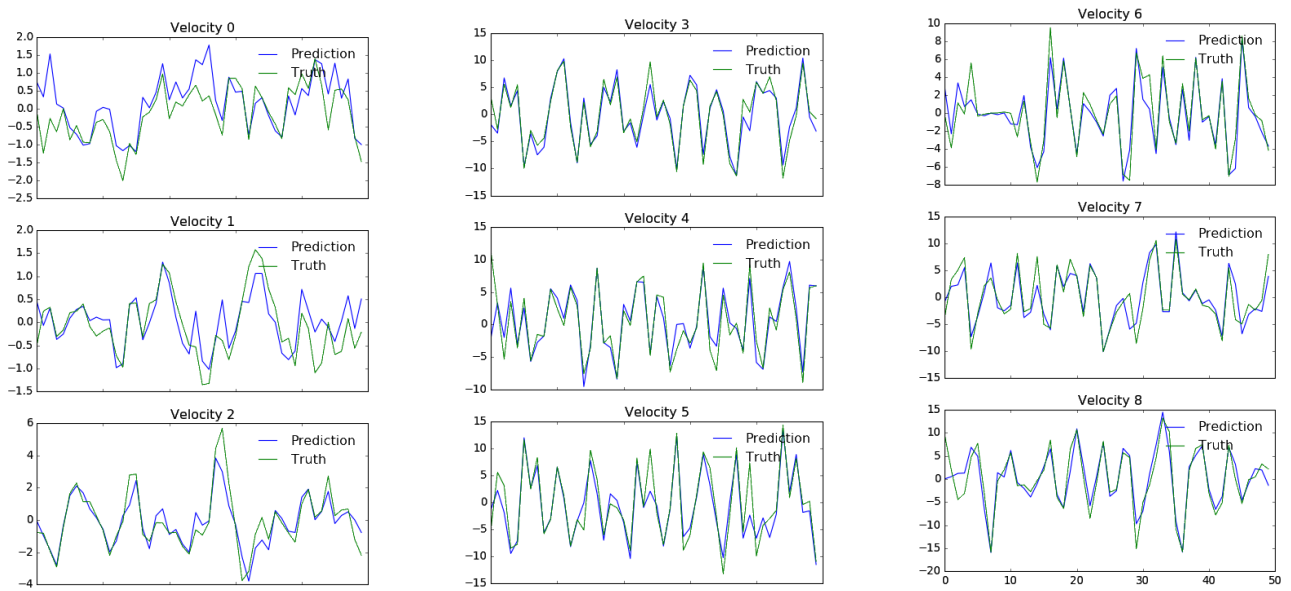


Figure 13: Given the Bayesian PlaNet agent, we predict the true velocities of the reinforcement learning environment from its encoded latent states. Compared to Figure 7 of [Hafner et al. \(2018a\)](#), Bayesian PlaNet appears to capture more information in the latent codes resulting in more precise velocity predictions (“world knowledge”).